

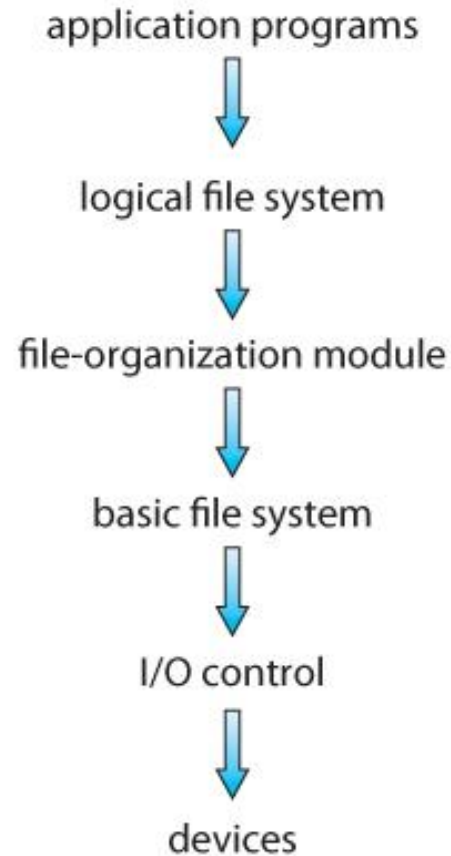
File System Implementation

Dr. Jit Mukherjee



File System Structure

- Suitable properties of Hard Disk -> Blocks of data can be rewritten in place
 - direct access, allowing any block of data to be accessed with only (relatively) minor movements of the disk heads and rotational latency.
- Disks are usually accessed in physical blocks, rather than a byte at a time.
 - 512 bytes to 4K or larger.
- File systems on disk drives layered design
 - lowest layer->physical devices. magnetic media, motors & controls, and the electronics connected and controlling them
 - *I/O Control* consists of *device drivers*, special software programs -> communicate with the devices by reading and writing special codes
 - *Basic file system* works directly with device drivers, retrieving and storing raw blocks of data, without knowing what is in block.
 - *File organization module* knows about files and their logical blocks, and how they map to physical blocks on the disk.
 - *logical file system* deals with meta data of a file (UID, GID, mode, dates, etc)-> everything about the file except the data itself.
 - Manages the directory structure and the mapping of file names to *file control blocks, FCBs*->contain all of the meta data and block number info for finding the data on the disk.

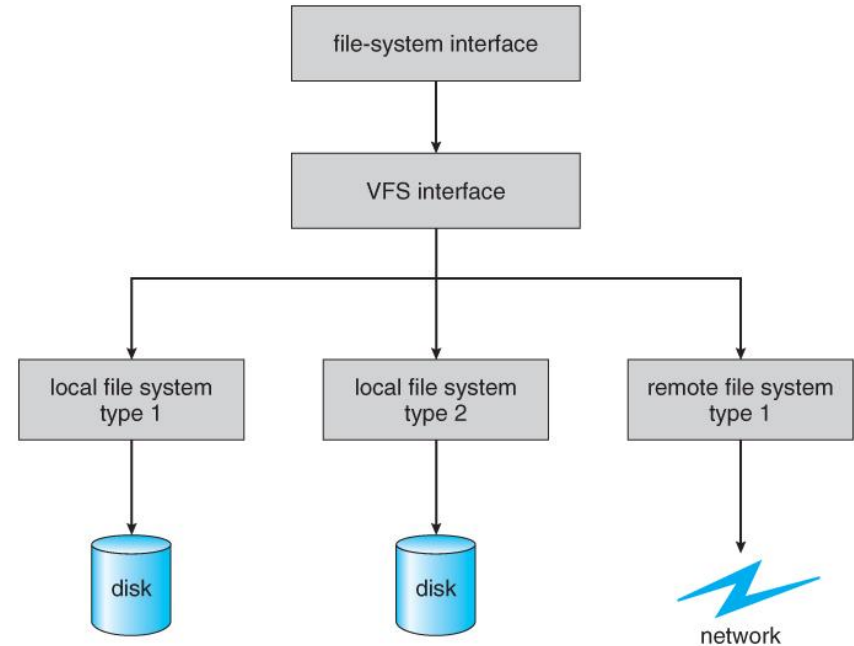


File System Implementation

- File systems store several important data structures on the disk:
 - Boot-control block (Boot block-Unix, Partition boot sector-Windows) - contains info about how to boot the system off.
 - Volume-control block (master file table-Unix, superblock-Windows) - contains info->partition table, number of blocks on each filesystem, and pointers to free blocks and free FCB blocks.
 - A directory structure->file names and pointers to corresponding FCBs->inode-Unix, Master file table-NTFS
 - File control block - details about ownership, size, permissions, dates, etc. UNIX-inode, Windows-rdbms
- Several key data structures stored in memory: in-memory mount table,
 - in-memory directory cache of recently accessed directory information,
 - a system-wide open file table (containing a copy of the FCB for every currently open file in the system),
 - A per-process open file table (a pointer to the system open file table as well as some other information)
- New file is created->a new FCB, all info regarding new file->The appropriate directory is modified with FCB
- File access->open() call reads CB from disk and store it in open file table
 - Entry in per process open file table-> index into per-process table, file descriptor-UNIX, file handle-Windows
 - Already opened->counter in system wide table increment, per-process table adjusted
 - File close->per-process table freed, counter - -, if needed clear cache

Partition and Mounting, Virtual File Systems

- Divide physical block->smaller units (partitions)
 - combine to larger, commonly for RAID installation
- Partitions->as raw devices (with no structure imposed upon them)
 - formatted to hold a filesystem (i.e. populated with FCBs and initial directory structures as appropriate.) - General users
 - Raw Partitions Used->swap spaces, certain databases, etc.
- Boot block -> raw partition. Modern system-> multiple OS, provides options
 - Continuing with boot process additional file systems get mounted
- Root Partition ->OS kernel, key portion of OS to complete booting
- VFS provides common interface to multiple different file systems
 - Unique identifier for files to access over all spaces
 - Linux VFS - inode (individual file), file (open files), superblock (represent filesystem), dentry (directory)
 - A few common functionalities, open, read, write

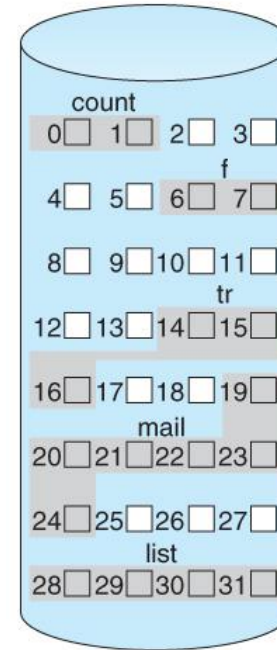


Directory Implementation

- Directories need to be fast to search, insert, delete->min wasted disk space
- Linear List - simplest and easiest dir structures to set up->drawbacks
 - Linear list of file names with pointer to data blocks
 - Finding a file or ifexists() -> linear search
 - Deletion -> moving all entries, flagging an entry as deleted, or by moving the last entry into the newly vacant position
 - Sorting -> search faster -> complex insertion and deletion
 - More complex data structures -> B tree can be considered
- Hash Table - Speed up the searches
 - Generally implemented on the top of Linear list
 - An efficient hash function
 - Collision -> Chained-overflow hash table

Allocation Methods: Contiguous

- Storing file on disks: all blocks of a file be kept together contiguously
 - Less movement of head->fast performance
- Similar issues with contiguous main memory allocation->fragmentation problem
 - Difference -> high time penalty
- Problem-> when file grow or exact size of the file is not known.
 - Overestimation -> external fragmentation
 - Underestimation->abort process, move file
 - If a file grows slowly over a long time period and the total final space must be allocated initially.
- A variation is to allocate file space in large contiguous chunks, called *extents*.
 - When a file outgrows its original extent, then an additional one is allocated.

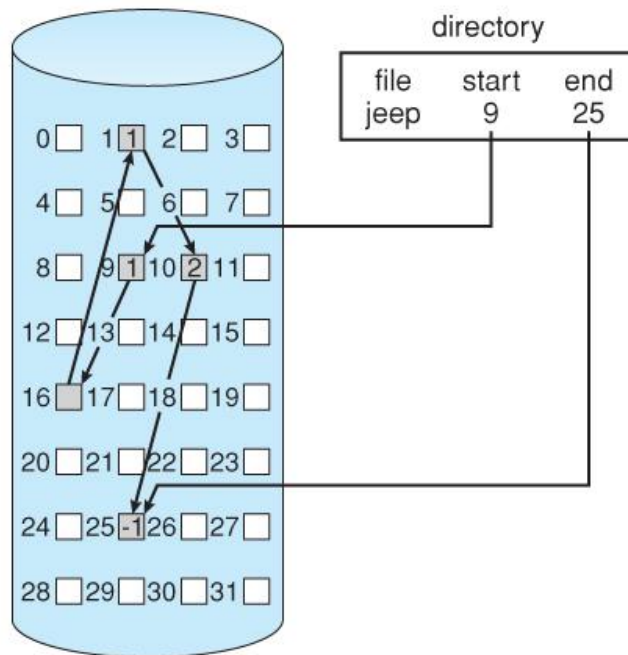
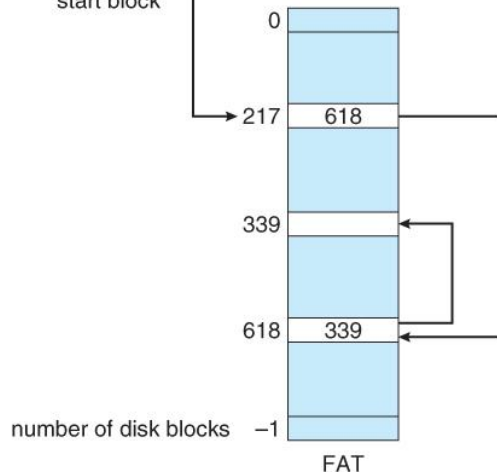
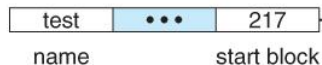


directory

file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

Linked Allocation

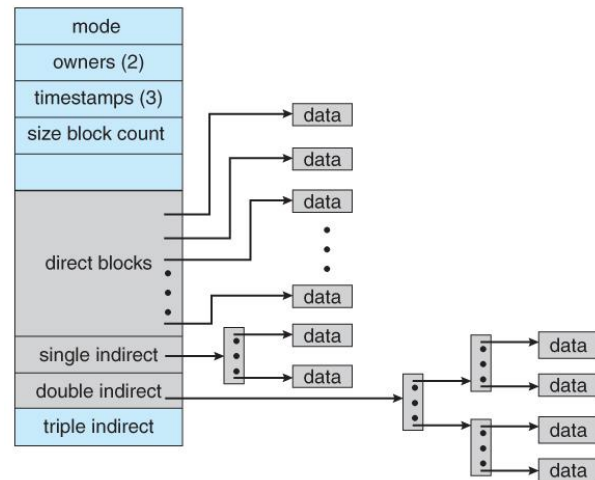
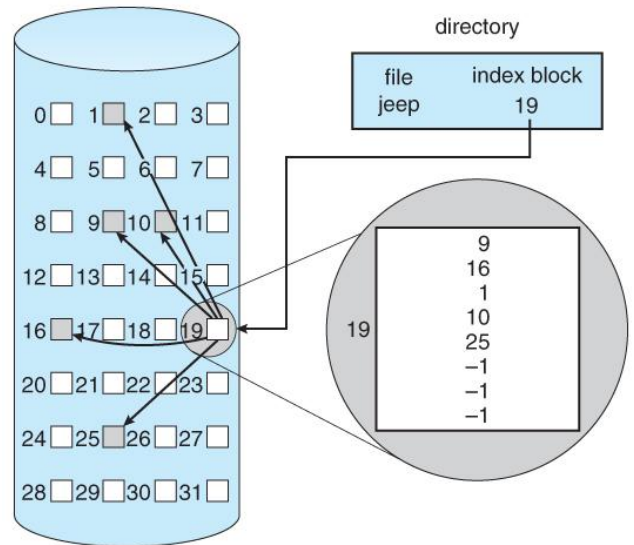
directory entry



- Disk files as linked list -> extra storage for the link
- No fragmentation -> no known filesize is required -> file can grow dynamically
- Efficient -> sequential access, !random access
- Cluster of block -> min internal fragmentation, less space for pointers
- If pointer get lost -> doubly linked list
- File allocation table, FAT variation -> can be cached, good for random access
 - Links -> separate table -> beginning of a disk

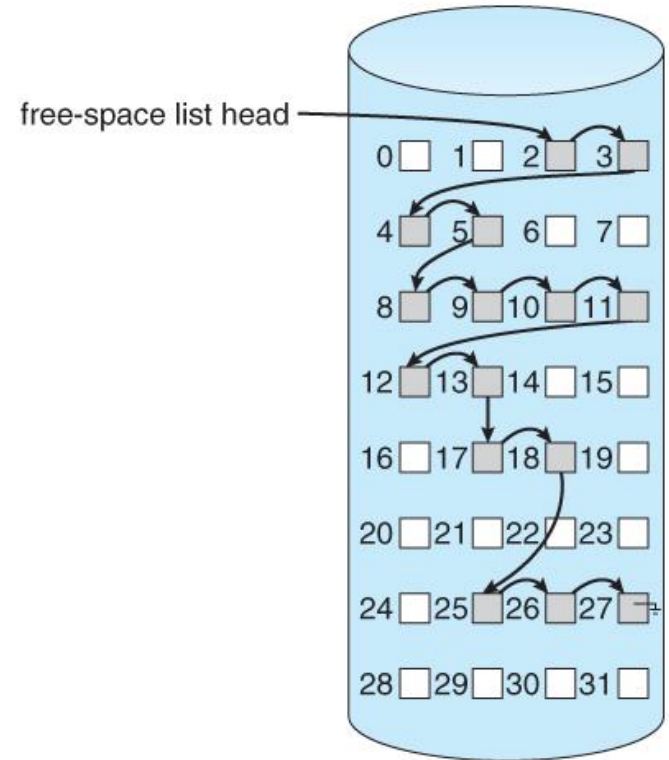
Indexed Allocation

- Linked allocation !good->direct access
 - The index block -> all pointers together
 - i-th entry-> i-th block of the file-> address
 - Some disk space is wasted -> index block
 - Less than linked or FAT
- Linked Scheme -> One index block=one disk block
- Large file-> link several index blocks
- Multilevel Index
- Combined Scheme -> Unix inode
 - First 12 blocks pointers directly in inode
 - Singly, doubly, and triply indirect pointers provide access to more data blocks
 - Small files -> Readily accessible
- Performance - Depends on random or direct access, large or small file. Support multiple



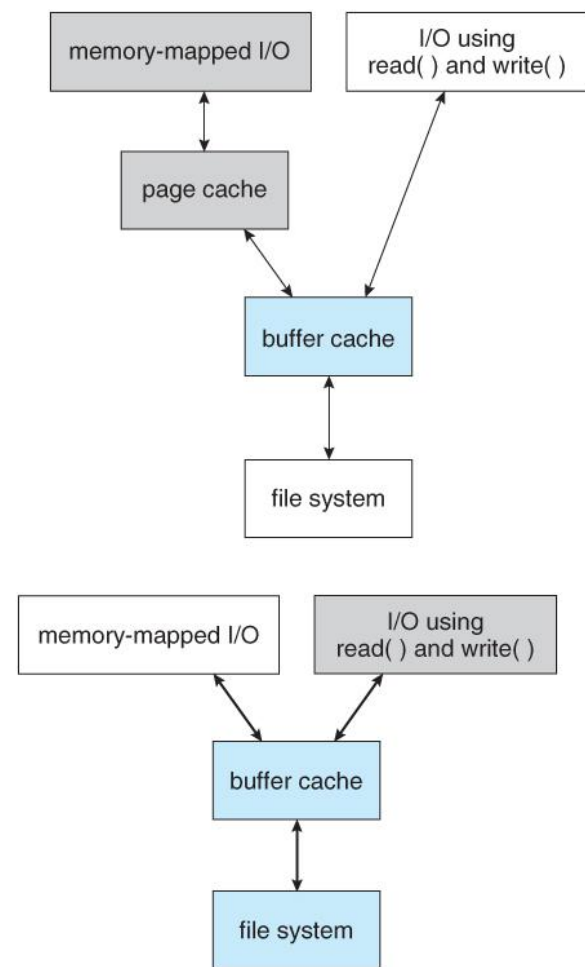
Free Space Management

- Keeping track of and allocating free space
- Bit Vector -> each bit represents a disk block, set to 1 if free or 0 if allocated.
 - Fast algo to find contiguous blocks
 - Size->40GB disk requires 5MB for bitmap
- Linked list->traversing or finding difficult
 - Fat table keep track of the free list
- Grouping->store address of first n free blocks in the first block
 - Last block-> another n free blocks
- Counting-> First free block and number n of free contiguous blocks



Efficiency and Performance

- Unix preallocate inodes->occupies space before any file created
 - Distribute inodes across disk->store nearby ata files->reduce disk seek distance
- Some system use various size clusters depending on filesize
- Kernel table size used to be fixed->changed rebuilding kernel
- Disk controller generally include on-board caching. Seek is needed
 - Head moved into place->entire track read->requested sector returned
 - Unrequested sector cached. Some OS cache disk block->buffer cache.
 - Page cache is more efficient. Caching for both process page and file data->unified buffer cache
- Priority Paging->process pages priority over file I/O page
 - neither can knock the other completely out of memory
 - LRU not good for sequential access files. **Free-behind**-frees up a page as soon as next page is requested. We may not need the page now. Read-ahead -> read requested page and subsequent pages
 - Asynchronous write schedules write more efficient manner. !metadata



Recovery, Backup and Restore

- In system crash->volatile memory structure lost, info disk->inconsistent
- Consistency Checker (fsck-UNIX, scandisk-Windows)->run at boot time or mount time, whenever filesystem not closed down properly
 - Checks->Disk blocks allocated to files or free list->Disk blocks allocated to neither->Disk blocks allocated to multiple files->number of disk block allocated to files inconsistent with file size->Properly allocated inodes which does not appear in directories
 - Link count for an inode not matching with number of references->identical files name
- Log-based-transaction-oriented (journaling) -> borrows from db
 - All metadata changes->logs. Similar to Transaction in process synchronizations
- Full backup->backups all files. Incremental->changes from previous time
 - A combination of full and incremental. Strategy->Month start->full. End of weeks ->inc
- Backup tapes should be tested regularly and kept separately (fires etc.)
- Storing important files in multiple computers

You guys have been a lovely audience.

Thank You!

All the best for the End Sem

- J.M.